

Climate ML Project — Full Context & Progress Log

This document captures all decisions, explanations, and progress from the project planning conversation. Use this to resume work if the original chat is lost or too large.

Project Overview

Goal: End-to-end data science portfolio project demonstrating Python + SQL + ML together.

Prediction target: Tomorrow's maximum temperature (regression) for Canadian weather stations.

Why this project:

- Mirrors real production data pipelines (not a notebook toy)
 - Combines data engineering + SQL feature engineering + ML + API serving
 - Strong portfolio signal: shows you can build a *system*, not just run a model
 - Domain credibility from ECCC background
-

Tech Stack

Layer	Tool	Why
Database	PostgreSQL 16 (Docker)	Industry standard, real schema design
Ingestion	Python + psycopg2	Parse GHCN files, bulk load
Feature engineering	SQL window functions	The key differentiator — not pandas
Modeling	XGBoost + scikit-learn	Industry standard for tabular regression
Serving	FastAPI	Lightweight, modern, easy to document
Dashboard	Streamlit	Fast to build, demo-able in interviews
Containerization	Docker Compose	One-command setup, portfolio signal

Data Source

NOAA GHCN-Daily (Global Historical Climatology Network)

- Public URL: <https://www.ncei.noaa.gov/pub/data/ghcn/daily/>
- Key files:
 - `readme.txt` — read this first, explains everything
 - `ghcnd-stations.txt` — fixed-width file, all stations worldwide
 - `ghcnd-inventory.txt` — which variables each station has, date ranges

- `by_station/<station_id>.csv` — one CSV per station, daily observations

Canadian stations: IDs starting with `CA` (e.g. `CA001010`)

Raw CSV format (no header row):

```
Column A = station_id      (11 chars, e.g. CA001010)
Column B = date             (YYYYMMDD, e.g. 19840701)
Column C = element          (TMAX, TMIN, PRCP, SNOW, SNWD)
Column D = value            (in tenths of units – see below)
Column E = mflag            (measurement flag, usually empty)
Column F = qflag            (quality flag – NULL = good data)
Column G = sflag            (source flag, C = Environment Canada)
Column H = obs_time         (usually empty)
```

Unit conversions (raw values are in tenths):

- TMAX, TMIN: divide by 10 → degrees Celsius (e.g. 256 = 25.6°C)
- PRCP, SNOW, SNWD: divide by 10 → millimetres (e.g. 32 = 3.2mm)

Quality flags (qflag): NULL means passed all checks. Any letter code means NOAA flagged it. Key bad flags: G (gap check fail), S (spatial inconsistency), X (bounds check fail — physically impossible value). We filter these out with `WHERE tmax_qflag IS NULL`.

Scope for this project:

- Region: Canada only
 - Stations: start with 50, expand to ~700
 - Date range: 20+ years of history
 - Variables: TMAX, TMIN, PRCP, SNOW, SNWD
-

Architecture



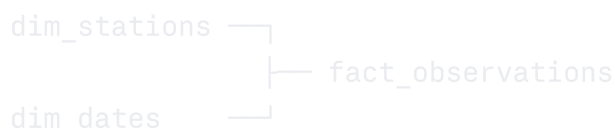
Database Schema (Star Schema)

Why PostgreSQL instead of just pulling from NOAA directly?

1. **Speed** — one SQL query vs downloading hundreds of CSVs
2. **Reliability** — NOAA has downtime; local DB doesn't
3. **Feature engineering** — pre-compute expensive rolling averages once
4. **Real-world mirror** — production systems never query raw sources directly
5. **Portfolio signal** — a system vs a script

Why a star schema?

Star schema = one central fact table surrounded by dimension tables. This is the standard data warehouse pattern used in industry.



ER Diagram

Entities and their attributes (from hand-drawn diagram):

dim_stations (PK: station_id)

- station_id (PK)
- name
- province
- latitude
- longitude
- elevation
- wmo_id
- first_year
- last_year

dim_dates (PK: date_id)

- date_id (PK)
- year
- month
- day
- day_of_year
- week_of_year
- quarter
- season
- is_leap_year

fact_observations (PK: station_id + date_id)

- station_id (FK → dim_stations)
- date_id (FK → dim_dates)
- tmax_c
- tmin_c
- prcp_mm
- snow_mm
- snwd_mm
- tmax_qflag
- tmin_qflag
- prcp_qflag

Relationships:

- dim_stations 1 → N fact_observations
- dim_dates 1 → N fact_observations

features_daily (materialized view — NOT in ER diagram)

A materialized view is a saved query whose results are physically stored, like a table, but with no primary/foreign keys. It is NOT an entity — that's why it doesn't appear in the ER diagram.

It is derived from fact_observations + dim_stations + dim_dates using SQL window functions. Refresh with:

sql

```
REFRESH MATERIALIZED VIEW features_daily;
```

Columns in features_daily:

- station_id, date_id (from fact_observations)
- year, month, day_of_year, season (from dim_dates)
- latitude, longitude, elevation_m (from dim_stations)
- tmax_c, tmin_c, prcp_mm, snow_mm (current day — known at prediction time)
- tmax_lag1, tmax_lag2, tmax_lag3 — yesterday's, 2 days ago, 3 days ago tmax
- tmin_lag1 — yesterday's tmin
- prcp_lag1 — yesterday's precip
- tmax_roll7 — 7-day rolling average tmax
- tmax_roll30 — 30-day rolling average tmax
- prcp_roll7 — 7-day rolling precip sum
- diurnal_range_lag1 — yesterday's (tmax - tmin)
- clim_tmax_mean — historical average tmax for this station + day_of_year
- clim_tmax_std — historical std dev of tmax
- tmax_anomaly — how much warmer/colder than normal right now
- **target_tmax_tomorrow** — what we're predicting (LEAD(tmax_c, 1))

Why SQL for feature engineering, not pandas?

- More efficient at scale — SQL handles large data before it touches Python
- Closer to how real ML pipelines work
- The key interview differentiator: most students do everything in pandas

Project Phases

Phase 0 — Setup (in progress)

- ☒ Folder structure created
- ☒ docker-compose.yml written
- ☐ Docker running successfully
- ☐ pgAdmin connected to climate_dw

- ☐ README stub created

Phase 1 — Data Acquisition

- ☐ Download ghcnd-stations.txt
- ☐ Filter to Canadian stations with 20+ years TMAX+TMIN
- ☐ Download 50 station CSVs to start

Phase 2 — Schema Design ☒ (designed, not yet written)

- ☒ ER diagram drawn on paper
- ☒ Entities and relationships identified
- ☐ schema.sql written and executed

Phase 3 — Ingestion Pipeline

- ☐ ingest.py written
- ☐ dim_stations loaded
- ☐ dim_dates populated (1900-2030)
- ☐ fact_observations loaded
- ☐ features_daily materialized view refreshed

Phase 4 — SQL Feature Engineering

- ☐ Verify lag features correct (lag1 = previous row's tmax)
- ☐ Verify rolling averages correct (spot-check against raw data)
- ☐ Verify climatology baseline makes sense

Phase 5 — Modeling

- ☐ Pull features_daily into pandas via single SQL query
- ☐ Chronological train/test split (NOT random)
- ☐ Train baseline (climatology average)
- ☐ Train XGBoost regressor
- ☐ Evaluate: RMSE, MAE, compare vs baseline

Phase 6 — FastAPI

- ☐ GET /predict?station_id=X&date=Y
- ☐ GET /stations (list available stations)

Phase 7 — Dashboard

- ☐ Streamlit app showing predictions vs actuals
- ☐ Station map
- ☐ Error metrics over time

Phase 8 — Portfolio Polish

- ☐ Architecture diagram in README
- ☐ ER diagram image in README (docs/er_diagram.png)

- ☐ Docker Compose one-command setup verified
 - ☐ Key results documented
-

Docker Setup

docker-compose.yml (current working version)

yaml

```
services:
  postgres:
    image: postgres:16
    container_name: climate_db
    restart: unless-stopped
    environment:
      POSTGRES_USER: climate
      POSTGRES_PASSWORD: climate123
      POSTGRES_DB: climate_dw
    ports:
      - "5433:5432"          # 5433 because local Postgres 18 uses 5432
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./sql/schema.sql:/docker-entrypoint-initdb.d/01_schema.sql

  pgadmin:
    image: dpage/pgadmin4
    container_name: climate_pgadmin
    restart: unless-stopped
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@climate.dev
      PGADMIN_DEFAULT_PASSWORD: admin123
    ports:
      - "5050:80"
    depends_on:
      - postgres

volumes:
  postgres_data:
```

Important: Port is `5433:5432` because a local PostgreSQL 18 is installed on this machine and already occupies port 5432.

Docker commands

bash

```
docker compose up -d      # start containers
docker compose down       # stop (data preserved)
docker compose down -v    # stop + delete all data (use carefully)
docker ps                 # check what's running
docker logs climate_db    # check Postgres logs
docker pull postgres:16   # force pull correct version
```

Known issue — Postgres 18 conflict

Error encountered: Docker pulled postgres:18 instead of postgres:16, causing a volume format incompatibility error.

Fix:

```
bash

docker compose down -v    # wipe old incompatible volume
docker pull postgres:16   # pull correct version
docker compose up -d      # start fresh
```

pgAdmin connection settings (desktop app)

Since pgAdmin is installed as a desktop app (not via Docker):

- Host: `localhost`
 - Port: `5433` (not 5432 — see port conflict above)
 - Maintenance database: `climate_dw`
 - Username: `climate`
 - Password: `climate123`
-

Project Folder Structure

```
climate-ml/
├── .env.example           # copy to .env, fill credentials
├── .gitignore            # excludes .env, data/, models
├── docker-compose.yml    # Postgres + pgAdmin containers
├── requirements.txt      # pinned Python dependencies
├── README.md             # project README (public-facing)
├── CONTEXT.md            # this file – full context log
├── docs/
│   └── er_diagram.png    # ER diagram image (add manually)
├── sql/
│   └── schema.sql        # CREATE TABLE + materialized view
├── ingestion/
│   ├── download.py       # fetch GHCN data from NOAA
│   └── ingest.py         # parse + load into Postgres
├── modeling/
│   └── train.py          # feature pull + XGBoost training
├── api/
│   └── main.py           # FastAPI prediction endpoint
├── dashboard/
│   └── app.py            # Streamlit visualization
└── data/
    ├── raw/
    │   ├── ghcn-stations.txt
    │   ├── ghcn-inventory.txt
    │   ├── canada_stations.csv
    │   └── daily/
    │       └── <station_id>.csv
```

Key Concepts Explained

Idempotency

Ingestion scripts use `ON CONFLICT ... DO UPDATE` (upsert) instead of plain INSERT. This means you can re-run the script safely without duplicating data. Standard practice in real data pipelines.

Window functions (SQL)

Used in `features_daily` to compute features that look back across rows:

- `LAG(tmax_c, 1) OVER (PARTITION BY station_id ORDER BY date_id)` → previous row's tmax for the same station
- `AVG(tmax_c) OVER (...ROWS BETWEEN 6 PRECEDING AND 1 PRECEDING)` → rolling 7-day average

Chronological train/test split

Time series data must be split by time, not randomly. If you split randomly, future data leaks into training and the model looks better than it really is. Split: train on earlier years, test on later years.

Baseline comparison

Always compare the ML model against a naive baseline. Here, the baseline is climatology (predict today = historical average for this day-of-year at this station). If the model can't beat climatology, it has no skill.

Materialized view vs table

- Table: stores raw data, has PK/FK constraints
 - Materialized view: stores results of a query, no constraints, refreshed on demand. Use when the computation is expensive and you want to pre-compute it rather than recalculate every query.
-

Resources

Schema design

- *Database Design for Mere Mortals* — Michael Hernandez (read first half)
- *The Data Warehouse Toolkit* — Ralph Kimball (chapters 1-3 for star schema)
- CMU Database Group YouTube (Andy Pavlo, first 4-5 lectures)

SQL (window functions, CTEs)

- Mode Analytics SQL Tutorial: <https://mode.com/sql-tutorial> (advanced section)
- pgexercises.com — Postgres-specific practice
- StrataScratch — DS interview-framed SQL problems

Career context

- Target roles: ML engineer or applied data scientist
- Key gap to close: actual model training + evaluation projects (current experience is strong on data engineering side)
- Next coop goal: role that touches model-building or experimentation